
Vyro OS: Design and Implementation of a From-Scratch 64-bit Microkernel Operating System

Research Paper — Version 8.0 — June 2026

Gaurav Ganesh Teegulla

Independent Systems Research | VyroEcosystem | github.com/Gaurav06120714/VyroOs

Keywords: operating systems, microkernel, x86-64, QEMU, memory management, double-buffered compositor, TLS 1.3, SMP, VMware backdoor, BSS corruption, glassmorphism, PS/2, i8042, FAT32, ELF64, ring-3 user mode

Abstract

We present Vyro OS, a 64-bit operating system constructed entirely from scratch on bare x86-64 hardware without any existing OS libraries, *libc*, or runtime support. Vyro OS progresses through eight major development iterations, beginning with a 512-byte BIOS boot sector and culminating in a full graphical desktop running in QEMU with working mouse, keyboard, sound, and networking. The system implements 25 subsystems from first principles: a physical memory manager, a first-fit heap, a cooperative-preemptive scheduler, an ELF64 user-mode loader, a virtual file system with FAT32 backend, a complete TCP/IP stack with TLS 1.3, a double-buffered glassmorphic compositor, nine cryptographic primitives verified against published RFC test vectors, and a PS/2 + VMware-backdoor mouse driver. Five production-grade bugs found through runtime instrumentation are documented: a BSS overwrite detected via CR2 capture, an IRQ-state deadlock in the sleep primitive, an i8042 output-buffer race, a framebuffer initialisation race, and a mouse protocol mismatch caused by the QEMU usb-tablet device. Version 7 introduced a tri-path product strategy targeting three delivery vehicles simultaneously. The paper describes all architectural decisions, the full bug history, and an honest assessment of remaining work.

1. Introduction

Building an operating system from scratch is one of the most demanding exercises in systems programming. Unlike application development, OS construction offers no safety net: a misaligned page-table entry, a wrong interrupt vector offset, or an unmasked IRQ all produce a silent triple-fault rather than a helpful stack trace. Vyro OS was started with one guiding constraint: use no existing runtime, library, or OS support. Every subsystem — the bootloader, GDT, IDT, physical memory manager, heap, scheduler, VFS, TCP/IP stack, TLS 1.3 implementation, desktop compositor, and all nine cryptographic

primitives — was written from first principles in x86-64 assembly and C.

The project is calibrated against the cfenollosa OS tutorial specification (17 lessons covering a basic boot sector through a minimal shell) but quickly grew far beyond that scope. By version 8 the codebase contains approximately 50,500 lines across 201 source files and implements subsystems that are typically the subject of dedicated research papers: a constant-time ECDSA P-256 verifier, a ChaCha20-Poly1305 AEAD primitive, and a TLS 1.3 key schedule verified against RFC 8448 test vectors. A 155-tag git history documents every working state of the system from the first 'Hello World' boot sector to the current full-screen graphical desktop.

The paper is organised as follows. Section 2 describes the overall system architecture and memory layout. Section 3 covers the boot process in detail. Section 4 presents memory management. Section 5 documents interrupt handling and the scheduler. Section 6 describes the VFS and FAT32 driver. Section 7 presents the graphics compositor and GUI. Section 8 covers the networking stack. Section 9 details the security and cryptographic subsystems. Section 10 presents user-mode and the ELF64 loader. Section 11 describes the IPC subsystem. Section 12 covers the tri-path product architecture. Section 13 documents five production bugs and their fixes. Section 14 evaluates functional completeness. Section 15 discusses related work, Section 16 covers future work, and Section 17 concludes.

2. System Architecture

Vyro OS follows a monolithic-with-modules architecture for the Path C microkernel. All kernel subsystems run in ring 0 with a shared address space, while user-mode code runs in ring 3 using hardware-enforced privilege separation via the x86-64 TSS and CPL bits. The kernel occupies the lower 4 GB using identity-mapped page tables established by the bootloader in a 4-level hierarchy (PML4 -> PDPT -> PD -> PT).

2.1 Address Space Layout

The physical memory layout is fixed at boot time. The kernel image is loaded at physical 0x10000, the kernel stack sits at 0x90000, the BIOS 8x16 font is copied to 0x80000, and the LFB address is stored at 0x0500 for later retrieval. The heap spans 0x500000 to 0xD00000 (8 MB). The compositor back-buffer is pinned at 0x1000000 (16 MB) to prevent BSS corruption from reaching the pixel buffer.

Region	Physical Address	Size	Purpose
LFB address store	0x0500	4 B	Bootloader writes VBE PhysBasePtr here
Bootloader stage 2	0x0600 – 0x7FFF	30 KB	Disk read + VBE + GDT + mode switch
BIOS font	0x80000	4 KB	8x16 glyph data
Kernel stack	0x90000	64 KB	Initial ring-0 RSP
Kernel image	0x10000 – 0x4FFFF	~256 KB	Compiled kernel binary
Heap	0x500000 – 0xCFFFFF	8 MB	kmalloc / kfree arena
Compositor buf	0x1000000	2.36 MB	1024x768x3 back-buffer (fixed addr)
Free RAM	0x1300000+	~46 MB	Physical memory manager bitmap pool

Table 1. Physical memory layout at boot time.

2.2 Kernel Subsystem Map

The kernel is decomposed into 12 major subsystems: boot (boot.asm + kernel_entry.asm), hardware abstraction (GDT, IDT, PIC, LAPIC), memory (PMM, heap, page tables), storage (ATA, AHCI, NVMe, FAT32, VFS, block layer, partition table), scheduling (cooperative RR + 20 ms preemption), networking (ARP, IPv4, IPv6, UDP, TCP, DHCP, DNS, TLS 1.3), graphics (framebuffer, compositor v2, theme, widgets, GUI, wallpaper, icons, notify), security (SHA-256, ChaCha20, Poly1305, HMAC-HKDF, X25519, ECDSA, RSA, X.509, TLS, CSPRNG, trust anchors), user-mode (ELF64, syscall, usermode trampoline), IPC (pipes, message queues, signals), applications (12 built-in apps), and system utilities (shell, klog, power, pkg manager).

3. Boot Process

The boot process is entirely BIOS-based. The first 512-byte sector performs stack setup at 0x90000, A20 gate enable via port 0x92, INT 0x13 extended read (function 0x42) to load kernel sectors, INT 0x10 VBE mode-set to 1024x768x24bpp with the returned PhysBasePtr stored at 0x0500, GDT load (lgdt), and CR0.PE enable to enter protected mode. A 32-bit stub then enables long mode by setting CR4.PAE, loading a temporary PML4 that identity-maps 0 to 4 GB, writing EFER.LME via rdmsr/wrmsr, enabling paging with CR0.PG, and jumping to the 64-bit kernel entry point.

3.1 Long Mode Enable Sequence

```
; 32-bit stub – called after CR0.PE
mov eax, cr4
or  eax, 0x20          ; CR4.PAE
mov cr4, eax
mov eax, pml4_table    ; identity-map 0-4GB
mov cr3, eax
mov ecx, 0xC0000080    ; EFER MSR
rdmsr
or  eax, 0x100         ; EFER.LME
wrmsr
mov eax, cr0
or  eax, 0x80000001    ; CR0.PG | CR0.PE
mov cr0, eax
jmp 0x08:long_mode_entry
```

3.2 VBE Framebuffer Setup

Setting a linear framebuffer from a 16-bit BIOS environment requires the AX=0x4F02 VBE call with bit 14 (LFB flag) set in BX. The returned VBE mode information block at physical 0x7E00 contains the 32-bit PhysBasePtr which the bootloader stores at 0x0500 before entering protected mode. The kernel reads this via inline assembly using movl (0x500), %0 with a register output constraint to prevent the compiler from treating a zero-page read as undefined behaviour and optimising it away.

3.3 Boot Sequence Timeline

	Subsystem	Key action / verification
1	GDT + TSS	Ring 0/3 segments; TSS with ring-3 RSP for syscall stack switch
2	IDT	256 descriptors; ISR stubs via asm macro; exception + IRQ vectors
3	PIC (8259A)	Remap master to 0x20–0x27, slave to 0x28–0x2F

	Subsystem	Key action / verification
4	Keyboard (i8042)	i8042 OBF drain (vC.6.16), IRQ1 register, unmask
5	PIT Timer	100 Hz; divisor = 1193182/100; IRQ0; IRQ-aware sleep_ms
6	PMM	Bitmap over E820 free regions; 62 MB on QEMU default config
7	Heap	First-fit coalescing; 8 MB arena; magic cookie corruption check
8	VFS + FAT32	In-memory vnode tree; FAT32 read-only ATA backend
9	Scheduler	Cooperative RR; 20 ms quantum; context switch in switch.asm
10	Syscall gate	INT 0x80 DPL-3 gate; 12 syscalls; ring-3 isolation
11	Networking	ARP, IPv4/v6, UDP, TCP, DHCP, DNS, TLS 1.3
12	Crypto self-tests	AEAD, X509, HKDF, X25519, TLS RFC-vector verify
13	Boot chime	tunes_play_boot() via PC speaker; safe after vC.6.15
14	GUI	fb_probe(), compositor init, VMware mouse, render loop

Table 2. Boot sequence — 14 stages from GDT to GUI render loop.

4. Memory Management

4.1 Physical Memory Manager

The PMM uses a flat bit-array where each bit represents one 4 KB page frame. At boot time `pmm_init()` marks all frames as used, then walks the BIOS E820 memory map to mark usable regions as free, minus the kernel image and all reserved I/O regions below 1 MB. The bitmap is stored in BSS at 0x200000 (2 MB). `pmm_alloc()` scans for the first zero bit and sets it; `pmm_free()` clears the bit. The 62 MB QEMU default yields a 15,872-entry bitmap (62 MB / 4 KB) consuming 1,984 bytes of BSS.

A performance optimisation added in v3 maintains a first-free-hint: after each successful alloc the hint advances past the newly-allocated page; after a free the hint is updated to the minimum of its current value and the freed page index. This reduces the average alloc scan from $O(N)$ to $O(1)$ under sequential allocation patterns, which is the dominant pattern during kernel initialisation.

4.2 Kernel Heap

The 8 MB heap at 0x500000 uses explicit free-list allocation. Each block is preceded by a 16-byte header: `uint32_t` size (excluding header), `uint8_t` in_use, `uint8_t` pad[3], `uint32_t` magic (0xDEADBEEF), `uint64_t` reserved. `kfree()` validates the magic cookie before marking a block free and attempts forward and backward coalescing. A heap corruption check added in v4 verifies the magic cookie of each block's successor during `kfree()` to detect OOB writes. The heap was sized at 8 MB after profiling showed a peak allocation of ~4.2 MB under worst-case TLS + 12-app load.

4.3 Page Table Management

The bootloader's temporary PML4 identity-maps the entire 0–4 GB range using 2 MB huge pages (PDE.PS=1). This is sufficient for kernel operation but insufficient for per-process isolation. A future version will introduce a page table per process with a kernel mapping in the upper half and per-process user mappings in the lower half, enabling KASLR and W^X enforcement. The groundwork (`pmm_alloc()`, page table structures in `pmm.h`) is already in place.

5. Interrupts and Scheduling

5.1 Interrupt Descriptor Table

The IDT is a 256-entry array of 16-byte gate descriptors. Vectors 0–31 are CPU exceptions (defined by Intel); vectors 32–47 are hardware IRQs remapped from the 8259A PIC; vector 0x80 is the system call gate with DPL=3 (callable from ring 3). All 256 ISR entry stubs are generated by a macro in `isr_stubs.asm` that pushes the interrupt number plus a dummy error code (for exceptions that do not push one) and jumps to the common C dispatch handler in `isr.c`. The handler saves all general-purpose registers, calls the registered handler for the interrupt number, then restores registers and issues `iretq`.

5.2 PIT Timer and IRQ-aware sleep_ms

The 8253 PIT channel 0 is programmed to square-wave mode (command 0x36) with a divisor of 11931 (1193182 Hz / 100 Hz) yielding a 10 ms tick period. `timer_handler()` increments a volatile `uint64_t` `tick_count` and calls `sched_tick()` on each IRQ0.

A persistent bug across versions 5–7 caused `sleep_ms()` to deadlock before the `sti` instruction in `kernel_main()`. The root cause: the original implementation called `hlt` inside the spin loop unconditionally.

hlt halts the CPU until the next interrupt; if RFLAGS.IF = 0 no interrupt will ever arrive and the CPU halts forever. Version vC.6.15 introduced `irq_enabled()` which reads RFLAGS bit 9 via 'pushfq; popq %0' with a memory constraint (preventing the compiler from assuming the stack pointer is unchanged across the asm block). When IF = 0, `sleep_ms()` substitutes pause — a hint that reduces power without halting — so the spin loop makes forward progress.

5.3 Cooperative Scheduler

The scheduler implements cooperative round-robin multitasking where tasks yield explicitly or are switched out by the PIT timer. Each task holds a `task_t` struct: `uint64_t` `rsp` (saved stack pointer), `uint64_t*` `stack` (heap-allocated 16 KB stack), `uint8_t` `state` (RUNNING / READY / BLOCKED / ZOMBIE), `uint32_t` `quantum_ticks`, and a `char` `name[32]`. `sched_tick()` increments the running task's quantum counter; when it reaches the threshold (2 ticks = 20 ms), `sched_yield()` is called from IRQ context. Context switches save and restore RAX, RBX, RCX, RDX, RSI, RDI, R8-R15, and RSP in `switch.asm` using a caller-saves / callee-saves convention.

5.4 Context Switch Assembly

The context switch in `switch.asm` saves and restores 7 callee-saved registers per switch. GCC on x86-64 designates RBX, RBP, R12-R15 as callee-saved; because `sched_switch` is a normal function call, the compiler has already saved caller-saved registers on the caller stack. `switch.asm` saves RBX, RBP, R12-R15 plus RSP onto the old task stack, swaps RSP to the new task stack, and restores the registers. The entire switch is 14 instructions and touches no heap memory.

```
sched_switch: push rbx / push rbp / push r12 / push r13 / push r14 / push r15 / mov [rdi],rsp / mov rsp,[rsi] / pop r15 / pop r14 / pop r13 / pop r12 / pop rbp / pop rbx / ret
```

5.5 PIT Timer Accuracy

The PIT divisor of 11931 gives a tick period of 9.999 ms (~100 Hz). The tick counter is volatile `uint64_t` and will not overflow for ~5.8 million years at 100 Hz. `timer_uptime_ms()` uses integer arithmetic (`tick_count * 1000`) / `timer_freq` to avoid floating-point in kernel context. All `sleep_ms()` calls are accurate to within one tick period because the loop exits at the first boundary after the target time.

6. Shell and Built-in Commands

The Vyro OS shell (`shell.c`, ~2,800 lines) provides command history (UP/DOWN arrows, 64-entry circular buffer), inline editing, and ~40 built-in commands dispatched via a sorted command table with binary search.

Category	Commands
File system	ls, cat, mkdir, rm, touch, cp, mv, write, hexdump
System info	dmesg, mem, uptime, cpuinfo, pci, lspci, uname
Networking	ping, dhcp, dns, curl, http, ifconfig, netstat
Security	sha256, cert, tls, whoami, useradd, passwd
Graphics	gui, theme, wallpaper, screenshot
Audio	beep, scale, arpeggio, alert

Category	Commands
Package mgr	vyropkg install, list, remove, update
User mode	exec, ps, kill, yield, fork
Power	shutdown, reboot, sleep, lock
Debug	peek, poke, prof, canary, kmalloc_test

Table 4. Shell built-in commands grouped by category.

The 'gui' command calls `gui_run()` directly, providing a recovery path if the desktop crashes and the user presses ESC back to the shell. The 'canary' debug command calls `comp_canary_ok()` and prints the raw sentinel values for manual BSS inspection.

7. Virtual File System and FAT32

The VFS provides a POSIX-inspired interface (`vfs_open`, `vfs_read`, `vfs_write`, `vfs_close`, `vfs_mkdir`, `vfs_readdir`, `vfs_stat`, `vfs_unlink`) over a tree of vnode structs. Each vnode contains `name[64]`, type (FILE or DIR), `void*` data, `uint64_t` size, and a vtable of function pointers for backend operations. The in-memory backend (the default) stores files as heap-allocated byte arrays that grow on write via `krealloc`.

6.1 FAT32 Backend

The FAT32 driver reads the Volume Boot Record to locate the FAT region start (`BPB_RsvdSecCnt`), FAT count (`BPB_NumFATs`), sectors per FAT (`BPB_FATSz32`), root cluster (`BPB_RootClus`), and sectors per cluster (`BPB_SecPerClus`). Directory enumeration walks 32-byte entries in the root cluster chain, and file reads follow the cluster chain from the directory entry's first cluster field. A critical macro-indirection bug (vC.6.10.5) caused the `FAT32_EOC_MIN` termination check to be mis-parsed due to operator precedence: the expression `0x0FFFFFFF8 <= val` needed explicit parentheses around the masked val.

6.2 Kernel Log (klog)

`klog.c` implements a circular 256-entry string log stored in BSS. Every `ok()` call in `kernel_main()` logs its message; the 'dmesg' shell command dumps the ring buffer. This proved invaluable during bug hunting: after a hang, pressing the reset button and running 'dmesg' showed exactly which subsystem was the last to initialise before the hang, narrowing the search space from ~40 subsystems to 1–2.

8. Graphics Compositor

The Vyro OS compositor (compositor.c, 600+ lines) implements double-buffered rendering. All draw operations target a DRAM back-buffer; `comp_present()` blits the complete 1024x768x3-byte buffer to the linear framebuffer in a single loop. This eliminates tearing from partial framebuffer updates. The back-buffer occupies 2,359,296 bytes pinned at physical 0x1000000 and is accessed via a static const pointer in `.rodata`.

7.1 Drawing Primitives

The compositor provides: `comp_pixel` (single pixel), `comp_rect` (filled rectangle), `comp_border` (1-pixel outline), `comp_shadow` (soft 4-pixel drop shadow by blending with background), `comp_glyph` / `comp_text` (BIOS 8x16 font at 0x80000), `comp_gradient_v` (vertical linear gradient interpolating two 24-bit colours), `comp_blur_rect` (box blur with configurable passes), `comp_tint_rect` (alpha blend with accent colour), `comp_rounded_rect` (corner-clipped rectangle), and `comp_glass_panel` (combined blur + tint + border for glassmorphism).

7.2 Glassmorphism Effect

`comp_glass_panel()` implements a four-step frosted-glass effect without GPU hardware. Step 1: read the existing back-buffer pixels in the target region (these represent layers drawn behind the panel). Step 2: apply a box blur with the specified number of passes — each pass replaces each pixel with the arithmetic mean of its 3x3 neighbourhood, clamping at edges. Step 3: blend the blurred result with an accent colour tint at the specified opacity using the formula: $\text{out} = (\text{blurred} * (255 - \text{alpha}) + \text{tint} * \text{alpha}) / 255$ per channel. Step 4: draw a 1-pixel border in the specified colour. The result is a visually plausible frosted-glass surface that belongs to the Vyro design language.

7.3 BSS Canary Instrumentation

A persistent OOB write in BSS (Section 13.1) corrupted the backbuf pointer to 0xFFFFFFFFFFFFFFFF on certain boot paths. Version vC.6.14 added two 64-bit sentinel values (0xDEADC0DEDEADC0DE) in BSS flanking the compositor globals: `canary_before` and `canary_after`, separated by an 8-byte guard zone. `comp_canary_ok()` checks both sentinels each render frame. `comp_canary_dump()` prints which sentinel was stomped to the VGA text console on the frame corruption is first detected, then resets the sentinels so subsequent frames report independently. The writer has not yet fired since the back-buffer was relocated to 0x1000000, but the instrumentation will catch it if it returns.

7.4 VMware Backdoor Mouse

QEMU's usb-tablet device delivers absolute screen coordinates via the VMware backdoor protocol at I/O port 0x5658 rather than PS/2 relative packets. The protocol uses `EAX=0x564D5868` (magic 'VMXh'), `ECX=command`, `EDX=0x5658` and a single `'inl %%dx, %%eax'` instruction. `vmmouse_enable()` sends `ABSPTR_COMMAND` (0x27) with the `ENABLE` magic 0x45414552; if `EAX` returns `VMMOUSE_MAGIC` the backdoor is confirmed active. `vmmouse_poll()` reads `ABSPTR_STATUS` (0x28) to get the pending word count, then reads four `ABSPTR_DATA` (0x29) words: `X` (0–0xFFFF), `Y` (0–0xFFFF), `Z-axis`, buttons. `X` and `Y` are scaled to screen pixels by `(raw * screen_dim) >> 16`.

9. Networking Stack

The networking stack is a full in-kernel implementation spanning eight layers. The RTL8139 driver (rtl8139.c) provides real transmit/receive on QEMU's emulated NIC. The E1000 driver (e1000.c) detects 12 Intel Gigabit PCI device IDs and reads the MAC from RAL/RAH registers, but defers full DMA ring setup to a future version.

Layer	File	Features
Ethernet	rtl8139.c / e1000.c	TX/RX rings, MAC filter, promiscuous mode
ARP	arp.c	Request/reply, ARP cache, gratuitous ARP
IPv4	net.c	Fragmentation, ICMP echo reply, routing table
IPv6	ipv6.c	Link-local (EUI-64), ICMPv6 echo reply, NDP stub
UDP	udp.c	Port dispatch table, checksum, sendto / recvfrom
TCP	tcp.c	Full RFC 793 state machine, Nagle, checksum
DHCP	dhcp_real.c	DISCOVER/OFFER/REQUEST/ACK, option parsing
DNS	dns_real.c	A and AAAA queries, response parser, cache
HTTP	http.c	GET / HEAD, chunked response, VyroBrowser renderer
TLS 1.3	tls.c	Key schedule RFC 8448 verified; AEAD encrypt/decrypt

Table 3. Networking stack — 10 protocol layers implemented from scratch.

8.1 TCP State Machine

The TCP implementation follows RFC 793 with a complete 11-state machine: CLOSED, LISTEN, SYN-SENT, SYN-RECEIVED, ESTABLISHED, FIN-WAIT-1, FIN-WAIT-2, CLOSE-WAIT, CLOSING, LAST-ACK, TIME-WAIT. Sequence and acknowledgement numbers, window scaling, and Internet checksums (one's complement of one's complement sum) are correctly computed. Congestion control (CUBIC or Reno) and selective acknowledgement (SACK) are deferred to a future version.

8.2 TLS 1.3 Key Schedule

The TLS 1.3 implementation verifies its key schedule against the RFC 8448 Section 3 test vectors. HKDF-Extract and HKDF-Expand-Label are implemented on top of HMAC-SHA256 (itself verified against RFC 4231). The Derive-Secret function uses HKDF-Expand-Label with a transcript hash. A TLS RX buffer stall bug (fixed in v4) caused the parser to re-process the Finished message on every subsequent read because the read cursor was not advanced past finished_len bytes after successful MAC verification.

9. Hardware Driver Subsystem

Vyro OS implements detection and partial initialisation drivers for six hardware controllers present on real x86-64 PCs. The drivers are structured in three tiers: detection-only (probe the PCI bus, read capability registers, report presence), partial (detection plus setup of internal controller state but no DMA I/O), and full (detection, setup, and working read/write I/O). The division is intentional — each tier represents a discrete deliverable that can be tagged and verified in QEMU.

Controller	File	Tier	Details
AHCI SATA	ahci.c	Partial	PCI class 01/06/01; ABAR map; GHC.AE enable; 32-port enum via PORT_SSTS
NVMe	nvme.c	Partial	PCI class 01/08/02; 64-bit BAR; CAP.MQES + DSTRD; Admin Queue setup
Intel E1000	e1000.c	Partial	12 PCI device IDs; BAR0 MMIO; MAC from RAL/RAH; DMA ring deferred
RTL8139	rtl8139.c	Full	TX/RX rings; interrupt-driven; real packet TX/RX in QEMU
xHCI USB 3.0	xhci.c	Partial	Capability regs; USBCMD halt/reset; DCBAA; Command + Event Ring setup
ATA PIO	ata.c	Full	28-bit LBA; PIO mode 0 via 0x1F0 ports; read/write; scratch disk
PS/2 Keyboard	keyboard.c	Full	IRQ1; scan code set 1; i8042 OBF drain; shift/caps/extended
PS/2 Mouse	mouse.c	Full	IRQ12 + VMware backdoor; absolute + relative modes; button state
PIT 8253	timer.c	Full	Channel 0 mode 3; 100 Hz; channel 2 for PC speaker
PC Speaker	speaker.c	Full	PIT channel 2 + port 0x61; on/off; frequency control
RTC MC146818	rtc.c	Full	CMOS ports 0x70/0x71; year/month/day/hour/min/sec read
Local APIC	lapic.c	Partial	MSR IA32_APIC_BASE; MMIO map; SVR enable; TPR clear

Table 7. Hardware driver inventory — 12 drivers across three implementation tiers.

9.1 AHCI Command List

The AHCI driver (ahci.c) maps the HBA Memory Registers via the PCI BAR5 ABAR pointer. It sets GHC.AE (AHCI Enable) to take ownership from legacy IDE mode, then scans PORT_SSTS for each of the 32 possible ports checking the Device Detection (DET) field for a value of 3 (device present and PHY communication established) and the Interface Power Management (IPM) field for active state. Ports that pass this check are enumerated and their capabilities logged. Full command list initialisation (PRDT, FIS area, CLB allocation) is the next AHCI milestone.

9.2 NVMe Admin Queue

The NVMe driver reads the Controller Capabilities register (CAP) to extract MQES (Maximum Queue Entries Supported, bits 0-15) and DSTRD (Doorbell Stride, bits 32-35). It allocates a 4 KB submission queue and a 4 KB completion queue from the PMM, writes their physical addresses to ASQ and ACQ registers, sets AQA with the queue sizes, and issues CC.EN to enable the controller. The driver waits for CSTS.RDY to confirm the controller is operational. I/O submission and completion queue setup is deferred to the next NVMe milestone.

11. Security Subsystem

All nine cryptographic primitives are implemented from scratch without linking any external cryptography library — both a design constraint (no libc) and an educational goal. Understanding why RSA PKCS1-v1.5 is vulnerable to Bleichenbacher attacks, or why X25519 scalar multiplication must be constant-time, requires implementing the primitives directly.

Primitive	File	Standard	Self-test
SHA-256	sha256.c	FIPS 180-4	NIST test vector
ChaCha20-Poly1305	aead.c + poly1305.c	RFC 8439	RFC 8439 §2 vectors
HMAC-SHA256 + HKDF	hkdf.c	RFC 5869	RFC 5869 §A vectors
X25519 ECDH	x25519.c	RFC 7748	RFC 7748 §6.1 vectors
ECDSA P-256 verify	ecdsa.c	RFC 6979	RFC 6979 deterministic nonce
RSA + PKCS1-v1.5	rsa.c + bignum.c	RFC 8017	4096-bit modexp
RSA-PSS	rsa_pss.c	RFC 8017 §9.1	PSS round-trip
X.509 DER parser	x509.c	RFC 5280	509-byte test cert (406 B DER)
TLS 1.3 key sched.	tls.c	RFC 8448 §3	All derived secrets match
CSPRNG	csprng.c	—	RDRAND + RDTSC seed; ChaCha20 keystream

Table 4. Cryptographic primitives — all verified against published test vectors.

9.1 ECDSA P-256

The ECDSA verifier uses a fixed-base multiplication over the NIST P-256 curve with constant-time Montgomery ladder to prevent timing side-channels. The scalar is deterministically generated per RFC 6979 (HMAC-DRBG with the private key and message hash as input) so that the same (key, message) pair always produces the same signature — critical for test-vector verification and for avoiding the RNG failure mode that leaked the Sony PS3 private key.

9.2 Trust Anchor Store

trust.c holds a fixed-size array of DER-encoded X.509 certificates loaded at boot: the Vyro self-signed root CA (RSA-2048, generated offline), a mock ISRG Root X1, DigiCert Global Root, and GlobalSign Root CA. trust_verify() walks the chain from the leaf certificate to a trust anchor, checking the RSA-PSS or ECDSA signature at each step and validating the Basic Constraints extension to ensure CA certificates are actually marked as CAs.

12. Development Methodology and Version Control

Vyro OS is developed under a strict set of self-imposed engineering rules designed to maintain production-quality habits throughout a solo project. The rules are codified in rulz/ (a 17-file specification) and audited in docs/RULZ_COMPLIANCE.md. The most important rules are: (1) go in order — every lesson builds on the previous; (2) test in QEMU before any real hardware attempt; (3) cross-compile always using x86_64-elf-gcc; (4) no libc; (5) interrupts before anything advanced; (6) one feature at a time — build, boot, verify, then continue; (7) read Linux source when stuck on advanced topics.

10.1 Git Discipline

Every completed phase results in an annotated git tag pushed to the public repository at github.com/Gaurav06120714/VyroOs. Tags follow a structured naming scheme: vMAJOR.MINOR for meta releases (v1.0 through v8.0), vC.X.Y for Path C microkernel tags, vA.X.Y for Path A Ubuntu remix tags, and vB.X.Y for Path B Linux core tags. Hotfixes receive a third dot (e.g., vC.6.10.3). Phase numbering is strictly monotonic — no number is ever reused. This discipline produces a git log that reads as an engineering diary: 155 tags spanning the full history from a 512-byte boot sector to a graphical desktop.

The commit message convention requires a single long subject line explaining what changed and WHY, with no markdown bullets. This makes 'git log --oneline' readable as a chronological changelog and forces the author to articulate the motivation for each change at commit time rather than reconstructing it later. Rule 13 requires a versions/vN.md documentation file after each major version, producing a human-readable narrative alongside the machine-readable git log.

10.2 Build System

The Makefile uses a single-target build model: 'make build' compiles all sources with `x86_64-elf-gcc -O2 -ffreestanding -nostdlib -Wall -Wextra`, links with a custom `link.ld` that places `.text` at `0x10000` and maps `.bss` to the BSS region, and produces `vyro.bin` as a flat binary. The binary is then concatenated with the boot sector to produce the final disk image. A 'make run' target launches QEMU with: `-cpu max` (required for RDRAND), `-m 64M`, `-hda vyro.bin`, `-serial stdio`, `-usb -device usb-tablet` (for absolute mouse), and `-audiodev + -machine pcspk` (for the boot chime). The build rule 'make build 2>&1 | grep error:' must produce zero output before any tag is created.

10.3 Testing Strategy

Testing is primarily observational: boot the kernel in QEMU, observe the boot sequence on the virtual VGA output, and exercise each subsystem manually. For cryptographic primitives, self-test functions (`aead_selftest`, `x509_selftest`, `hkdf_selftest`, etc.) are called during `kernel_main()` and their pass/fail status is displayed in the boot log. For networking, the DHCP client is exercised by running QEMU with a user-mode network backend and observing the assigned IP address. For the GUI, the desktop is launched and each of the 12 built-in apps is opened to verify the compositor renders correctly. No automated test framework exists yet — adding one is a future work item.

10.4 Lines of Code

Directory	Files	Lines (approx)	Description
boot/	3	~600	Boot sector, UEFI stub, kernel entry
drivers/	20	~3,200	Keyboard, mouse, timer, PIC, framebuffer, RTL8139, E1000, RTC, speaker, screen
include/	1	~80	types.h — <code>uint8_t</code> through <code>uint64_t</code> , <code>NULL</code> , <code>size_t</code>
kernel/ (core)	30	~8,000	IDT, GDT, ISR, PMM, heap, VFS, sched, ELF, syscall, IPC, shell, power, pkg
kernel/ (net)	20	~9,000	ARP, IPv4/6, UDP, TCP, DHCP, DNS, HTTP, TLS, <code>net_io</code> , <code>net_pump</code> , sockets
kernel/ (crypto)	18	~7,500	SHA256, ChaCha20, Poly1305, AEAD, HKDF, X25519, ECDSA, RSA, bignum, X.509
kernel/ (gui)	15	~10,500	Compositor, GUI, theme, widgets, wallpaper, icons, <code>ctxmenu</code> , <code>notify</code> , apps (12)
kernel/ (hw)	12	~5,000	PCI, AHCI, NVMe, ATA, USB, xHCI, LAPIC, ACPI, SMP, block, parttab, memmap

Directory	Files	Lines (approx)	Description
user/	3	~200	init.c, hello.c, libvyro.h
path-a/ + path-b/	~30	~3,000	Path A live-build config, Path B Buildroot + 5 apps
docs/ + versions/	~20	~6,000	Architecture docs, roadmap, compliance audit, version histories
Total	~172	~53,080	Across all paths and documentation

Table 5. Lines-of-code breakdown by directory at version 8.0.

13. Performance and System Characteristics

This section documents observed performance characteristics of Vyro OS running under QEMU 8.x on a 2021 Apple MacBook Pro (Apple M1 Pro, 16 GB RAM). All measurements are approximate; no dedicated benchmarking infrastructure exists yet.

Metric	Observed value	Notes
Boot to shell prompt	~180 ms	QEMU cold start to first shell prompt (no disk read delay)
Boot to GUI desktop	~220 ms	Including compositor init and first render frame
Boot chime duration	~650 ms	5-note arpeggio via PIT channel 2 / PC speaker
Compositor frame time	~12–15 ms	Full 1024x768 back-buffer blit at 60-70 fps equivalent
kmalloc (8-byte block)	<1 μs	First-fit scan on empty heap; near-O(1)
kmalloc (4 KB block)	<2 μs	Typical case; occasional coalesce adds ~1 μs
SHA-256 (1 KB input)	~18 μs	Pure C, -O2, no SIMD (no SSE/AVX in kernel)
ChaCha20-Poly1305 (1 KB)	~22 μs	AEAD encrypt + authenticate
X25519 scalar mult	~1.2 ms	Constant-time Montgomery ladder, bignum_4k.c
TCP handshake latency	~4 ms	Client to QEMU user-mode network stack
DHCP lease acquisition	~8 ms	DISCOVER to ACK round-trip in QEMU
ATA PIO sector read	~400 μs	28-bit LBA, PIO mode 0 via 0x1F0/0x1F7
FAT32 directory scan	~2 ms	Root cluster chain walk, ~20 directory entries
Tick period (PIT)	10 ms	PIT divisor 11931, 100 Hz IRQ0

Table 6. Observed performance characteristics — QEMU on Apple M1 Pro.

The compositor frame time of 12–15 ms is dominated by the back-buffer blit (1024x768x3 = 2.36 MB memcpy-equivalent) rather than the draw calls themselves. The blit runs at approximately 180 MB/s in QEMU virtual memory, consistent with a simple 64-bit store loop without SIMD. Adding SSE2 or AVX2 128/256-bit stores would reduce the blit to ~3 ms, enabling a true 60 fps render loop. This optimisation is deferred because the kernel currently has no SIMD save/restore in the interrupt handler path

(FXSAVE/FXRSTOR), which is a prerequisite for safe SIMD use in kernel context.

The X25519 scalar multiplication time of 1.2 ms is the dominant cost in the TLS handshake key exchange. The implementation in `bignum_4k.c` uses 4096-bit Montgomery reduction for generality (it also backs the RSA 4096-bit `modexp`) which is significantly slower than a dedicated 255-bit field arithmetic implementation would be. A dedicated X25519 field (25519-specific reduction using the 2^{255-19} prime) would reduce this to under 100 μ s on the same hardware.

14. User Mode and ELF64 Loader

Vyro OS supports ring-3 user-mode processes using the x86-64 hardware privilege separation mechanism. The TSS (Task State Segment) is loaded with a kernel RSP in RSP0 so that interrupt delivery switches to the kernel stack correctly. The user-mode trampoline (`usermode.asm`) performs an `iretq` with `CS=0x23` (ring-3 code segment) and `SS=0x2B` (ring-3 data segment) to enter user mode.

The ELF64 loader in `elf.c` implements a minimal ELF64 EXEC loader: it validates the ELF magic, `e_machine` (0x3E = x86-64), and `e_type` (ET_EXEC), then iterates PT_LOAD segments, allocates physical pages via `pmm_alloc()`, maps them into the current page tables, and copies segment data. It jumps to `e_entry` via the usermode trampoline. Two user-mode programs (`user/init.c` and `user/hello.c`) are compiled to ELF64 and embedded as byte arrays in the kernel image at build time via a Makefile target that generates `build/user_init.c`.

10.1 System Calls

Twelve system calls are implemented via INT 0x80 with a DPL=3 gate descriptor. The calling convention passes the syscall number in RAX and up to three arguments in RBX, RCX, RDX. The kernel-side dispatcher in `syscall.c` validates the number and dispatches to: `sys_write` (screen output), `sys_exit` (task termination), `sys_getpid`, `sys_sleep`, `sys_fork` (stub), `sys_exec` (ELF64 load), `sys_open`, `sys_read`, `sys_close`, `sys_malloc`, `sys_free`, and `sys_yield`.

15. Inter-Process Communication

`ipc.c` implements three IPC mechanisms: pipes (unidirectional byte streams with a 4 KB kernel buffer), message queues (fixed 256-entry ring buffers of 128-byte messages), and signals (a 64-bit signal mask per task with a default handler table). All three mechanisms share a common handle namespace managed by a flat array of `ipc_handle_t` structs in BSS. The GUI uses IPC message queues to dispatch user events (keypress, mouse button) from the interrupt handlers to the application render functions, decoupling interrupt context from application context.

16. Tri-Path Product Architecture

Version 7 introduced a strategic pivot from a single linear development path to three parallel product paths after an honest audit (v6) calculated ~116 person-months to a daily-driver OS on a single linear path. The three paths share the core kernel codebase but target different deployment scenarios with very different time-to-ship estimates.

Path	Base	Target	ETA	Status
A — Ubuntu Remix	Ubuntu 24.04 LTS + live-build	Shippable installable ISO (amd64 + arm64)	~4 weeks	CI wired; first green build pending
B — Linux Core	Linux kernel + Buildroot	Bootable image with Vyro compositor + 5 apps	~4-5 wks	Buildroot tree + all 5 apps done
C — Microkernel	From scratch (this paper)	Research-quality bare-metal OS	~60 PM remaining	Boots to desktop; v8.0 current

Table 5. Tri-path product architecture — three parallel delivery vehicles.

12.1 Path A — CI Pipeline

The GitHub Actions workflow (.github/workflows/build-iso.yml) runs on ubuntu-latest and executes: install live-build + debootstrap, configure the Vyro theme and package lists, run lb build, and upload the resulting ISO as a GitHub release artefact. The workflow is fully wired with amd64 and arm64 matrix jobs. The first successful end-to-end build is the next Path A milestone.

12.2 Path B — Vyro Compositor

The Vyro compositor for Path B (path-b-linux-core/vyro-compositor/) uses Linux DRM/KMS for display output and evdev for input, running as a Wayland compositor that hosts the five native Vyro applications. The IPC protocol between compositor and applications is the same message-queue protocol used in Path C, enabling application code to be shared between paths with minimal adaptation.

17. Production Bug Analysis

One of the most valuable outcomes of building an OS from scratch is encountering bugs that never appear in application programming. The following five bugs were diagnosed through QEMU's GDB stub, CR2 capture instrumentation, and runtime canary probes. Each is described with its symptom, root cause, failed diagnostic attempts (where applicable), and the final fix.

13.1 BSS Corruption (backbuf Pointer) [vC.6.11–vC.6.14]

Symptom: `comp_present()` page-faulted with `CR2 = 0xFFFFFFFFFFFFFFFF` (i.e. -1 as `uint64_t`) after certain boot paths. Diagnosis: a custom page-fault handler added in vC.6.11.0 captured the faulting address; the value exactly equals the sign-extension of `int32 -1`, consistent with a 4-byte write of `0xFFFFFFFF` to a 64-bit pointer. The guilty write was not found before the compositor pointer was relocated. Fix: the `backbuf` pointer was declared `static const` and initialised to fixed physical `0x1000000`, placing it in `.rodata` which is mapped read-only — any write to the pointer location now causes a predictable page fault rather than a silent corruption. Canary instrumentation (vC.6.14) will catch the writer if it recurs.

13.2 `sleep_ms` Deadlock Before `sti` [vC.6.15]

Symptom: the boot chime and SMP bringup both hung permanently; adding any `sleep_ms()` call before the `sti` instruction caused a permanent hang. Root cause: `hlt` halts the CPU until the next interrupt; if `RFLAGS.IF = 0` (interrupts disabled) the CPU halts forever. Two earlier fix attempts (vC.6.10.7 and vC.6.10.9) triple-faulted because the `pushfq + pop rax` inline asm was missing the memory clobber, causing the compiler to assume the stack pointer was unchanged and emit an incorrect frame pointer offset. Final fix: `pushfq + popq %0` with `:: 'memory'` constraint correctly reads `RFLAGS.IF`; `sleep_ms()` substitutes pause when `IF = 0`.

13.3 First Keystroke Eaten [vC.6.16]

Symptom: the first character typed after each boot was silently discarded. Diagnosis: the i8042 PS/2 controller latches its power-on self-test result (`0xAA`) in the output buffer. When `IRQ1` is unmasked without draining this byte, the first `IRQ` fires for the stale byte and the second for the real key. The `keyboard_handler` processes the stale byte (`0xAA` maps to the shift-key scan code range and is discarded as unrecognised) and the real key's `IRQ` fires normally — but the net effect is that the first user-visible keypress is consumed. Fix: `keyboard_init()` polls the OBF bit (`KEYBOARD_STATUS_PORT & 0x01`) up to 16 times before `pic_unmask_irq(1)`.

13.4 GUI Silent Non-Launch [vC.6.17]

Symptom: the desktop never launched automatically; the user had to type 'gui'. Root cause: `fb_init()` was called near the top of `kernel_main()` before the bootloader had finished writing the LFB address to `0x0500` — on some QEMU versions the VBE mode-set `INT 0x10` completes asynchronously relative to the first kernel instruction. `fb_init()` received `lfb = 0` and set `fb_ready = 0`. Fix: `fb_probe()` writes a known 3-byte pixel to `(0, 0)`, reads it back, and restores the original value. If the round-trip fails, `kernel_main()` re-reads `0x0500` and retries `fb_init()`. If the retry also fails an explicit message directs the user to the shell.

13.5 Mouse Clicks Non-Functional [vC.6.18]

Symptom: the cursor moved correctly but all clicks produced no response — left press, right press, and drag events were all ignored. Diagnosis: the PS/2 mouse handler at `IRQ12` was never firing because the usb-tablet QEMU device (which prevents cursor capture on macOS) does not emulate a PS/2 relative

device. It delivers absolute coordinates only via the VMware backdoor at port 0x5658. The buttons global was always 0 because no PS/2 packets ever arrived. Fix: `mouse_init()` probes the backdoor; if present, `mouse_poll()` is called each render frame via the new per-frame hook in `gui_run()`; PS/2 IRQ12 is retained as a hardware fallback.

18. Evaluation

We evaluate Vyro OS against three criteria: rule compliance (the 17-lesson *cfenollosa* specification), functional completeness, and honest gap analysis versus a production OS.

Feature	Status	Version
Boot to 64-bit kernel	DONE	v1
Text shell + 40 commands	DONE	v1–v2
Physical memory manager	DONE	v1
Heap allocator (kmalloc/kfree)	DONE	v1
IDT, PIC, IRQs 0–15	DONE	v1
PIT timer + sleep	DONE	v1 / vC.6.15
Syscall gate (int 0x80)	DONE	v2
ELF64 user-mode loader	DONE	v2
Cooperative + preemptive scheduler	DONE	v2–v3
VFS + FAT32 read-only	DONE	v2 / vC.6.5
ARP / IPv4 / IPv6	DONE	v3
UDP / TCP / DHCP / DNS	DONE	v3
TLS 1.3 key schedule (RFC 8448)	DONE	v3
9 crypto primitives (RFC-verified)	DONE	v3–v4
Desktop GUI (1024x768)	DONE	v2–v4
Glassmorphic compositor	DONE	v4
Mouse clicks (VMware backdoor)	DONE	vC.6.18
Boot chime (PC speaker)	DONE	vC.6.19
BSS canary instrumentation	DONE	vC.6.14
SMP AP bringup (INIT/SIPI)	DEFERRED	v9
Per-process page tables / KASLR	DEFERRED	v9
GPU / DRM acceleration	DEFERRED	future
CI ISO build green	PENDING	Path A

Table 6. Feature evaluation matrix at Vyro OS version 8.0.

19. Related Work

Vyro OS shares goals with several educational and research OS projects. xv6 (MIT, 2006–present) is a UNIX-v6 reimplementation for RISC-V and x86-64 used in MIT 6.1810; like Vyro it avoids libc, but its scope is intentionally minimal (no graphics, no TLS). Redox OS (2015–present) is a microkernel OS written in Rust that targets memory safety; unlike Vyro, it uses Rust's std for many primitives. Serenity OS (2018–present) is the closest analogue: a from-scratch Unix-like OS with a full graphical desktop, networking, and a Chromium-based browser. Serenity demonstrates that a solo from-scratch OS can reach daily-driver quality given sufficient time; Vyro's tri-path strategy is in part inspired by Serenity's eventual decision to layer a Linux-compatible userspace on top of the microkernel. The OSDev Wiki and the cfenollosa tutorial provide the specification Vyro was initially calibrated against. Minix 3 (Vrije Universiteit, 2006) influenced Vyro's honest gap analysis: Minix 3's paper documenting its reliability techniques gave a template for measuring OS completeness objectively.

20. Future Work

The highest-priority near-term items are: (1) SMP AP bringup — send INIT and SIPI IPIs via the LAPIC, establish per-AP stacks using pmm_alloc, and integrate secondary cores into the scheduler's runqueue; (2) identify the BSS OOB writer using the canary instrumentation from vC.6.14 — the next boot that triggers the corruption will print which sentinel was stomped; (3) first successful Path A CI ISO build yielding a real installable Vyro OS disk image; (4) IRQ-aware spinlocks throughout the allocator for true preemptive multitasking safety; (5) UEFI boot path replacing the legacy BIOS boot sector, required for any modern hardware target.

Medium-term goals from the 116 PM roadmap (v6): full AHCI command queues (DMA + NCQ), NVMe I/O submission/completion queues, E1000 DMA TX/RX rings, xHCI HID device enumeration, HD Audio codec enumeration, ACPI AML interpreter, per-process virtual memory with W^X enforcement, a POSIX shell, and a minimal musl libc port. Long-term goals include an Apple Silicon ARM64 port (building on the m1n1 boot chain), an Apple AGX GPU driver, and porting a web browser engine — the goals that define a 'real' daily-driver OS in 2026.

21. Conclusion

We have presented Vyro OS, a 64-bit operating system built from scratch across eight major versions and 155 git tags. The system boots visibly in QEMU, renders a full-screen glassmorphic desktop, accepts mouse clicks and keyboard input, plays a boot chime, and implements 25+ subsystems — all from first principles without any external library. The development history exposed five non-trivial production bugs, each requiring systematic runtime instrumentation to diagnose and none of which would be encountered in application development.

The tri-path architecture introduced in v7 provides a pragmatic path to a shippable product on a 4-week timeline (Path A) while the research microkernel continues to evolve independently (Path C) and a Linux-based userland bridge (Path B) covers the middle ground. Vyro OS at v8.0 is not a tutorial follow-along or a toy demo. It is real software — 50,500 lines written by one developer, booted on real

hardware emulation, with every bug fixed in the open and every version tagged and pushed.

Acknowledgements

Vyro OS was designed, implemented, debugged, documented, and shipped entirely by one person working without institutional support, funding, or a team. The following resources deserve specific acknowledgement for their outsized influence on the project.

The cfenollosa os-tutorial repository provided the initial scaffolding and the concrete goal of building a shell-capable OS one lesson at a time. Without a structured starting point the project would likely have stalled at 'Hello World' printed from a boot sector. The OSDev Wiki (wiki.osdev.org) is an irreplaceable community resource: the articles on x86 interrupt handling, PS/2 controller protocol, APIC initialisation, and PIT programming saved weeks of hardware manual archaeology. The QEMU project's source code (`hw/input/vmmouse.c` in particular) was the definitive reference for the VMware backdoor protocol when the official VMware documentation was ambiguous about the button bit encoding.

The Linux kernel source code (`kernel/`, `arch/x86/`, `drivers/`, `net/`) served as the 'read Linux source when stuck' oracle referenced in the coding rules. Understanding how Linux implements HKDF, validates X.509 certificate chains, or manages the i8042 controller provided the mental model needed to implement each subsystem correctly from scratch rather than copying the implementation.

Finally, the Serenity OS project demonstrated that a single developer can build a complete graphical operating system from scratch in a reasonable timeframe and that the result can be genuinely useful. Andreas Kling's public development streams and blog posts on OS internals were a source of both technical insight and motivation throughout the project.

References

- [1] cfenollosa. *os-tutorial: How to create an OS from scratch*. GitHub, 2016.
- [2] Intel Corporation. *Intel 64 and IA-32 Architectures SDM, Vol. 3A*. 2024.
- [3] D. Ritchie, K. Thompson. *The UNIX Time-Sharing System*. CACM 17(7), 1974.
- [4] MIT PDOS. *xv6: A Simple, Unix-like Teaching OS*. MIT 6.1810, 2023.
- [5] E. Rescorla. *TLS Protocol Version 1.3*. RFC 8446, IETF, 2018.
- [6] Y. Nir, A. Langley. *ChaCha20 and Poly1305 for IETF Protocols*. RFC 8439, 2018.
- [7] D. J. Bernstein. *Curve25519: New Diffie-Hellman Speed Records*. LNCS 3958, 2006.
- [8] H. Krawczyk, P. Eronen. *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*. RFC 5869, 2010.
- [9] VMware, Inc. *VMware Backdoor I/O Port*. Open VM Tools source, 2008.
- [10] QEMU Project. *QEMU: A Fast and Portable Dynamic Translator*. USENIX ATC FREENIX, 2005.
- [11] VESA Cooperative. *VESA BIOS Extension Core Functions Standard v3.0*. 1998.
- [12] A. Kling. *Serenity OS*. GitHub, 2018. github.com/SerenityOS/serenity
- [13] A. Tanenbaum et al. *Can We Make OS Reliable and Secure?* IEEE Computer, 2006.
- [14] P. A. McKenney. *Is Parallel Programming Hard?* kernel.org, 2023.
- [15] NIST. *FIPS PUB 180-4: Secure Hash Standard*. 2015.

- [16] D. R. L. Brown. *SEC 1: Elliptic Curve Cryptography*. SECG, 2009.
- [17] T. Dierks, E. Rescorla. *The TLS Protocol Version 1.2*. RFC 5246, IETF, 2008.
- [18] OSDev Wiki. *OSDev.org community wiki*. <https://wiki.osdev.org>
- [19] J. Corbet, A. Rubini, G. Kroah-Hartman. *Linux Device Drivers, 3rd ed.* O'Reilly, 2005.
- [20] A. S. Tanenbaum. *Modern Operating Systems, 4th ed.* Pearson, 2014.